Tender RAG 2.0 and MultiDoc Evidence Agent

Analysis, project plan and job-market targeting

Two connected AI-engineering portfolio projects

Period: November 2026 to January 2027 · about 6 hours per week (≈ 72 h)

Applications to AI / GenAI / Agentic AI Engineer roles, Germany and remote

Prepared for: Shalin Vachheta

Status: plan. Part A describes the code as READ, not as run. Nothing in Parts B and C has been built or measured yet. Every target marked TBD is set from the baseline measurement, never guessed.

Version 1.0 · November 2026 · repository: multilingual-tender-rag (branch chore/repo-cleanup)

Contents

A1 Executive summary 4

A2 The problem and the business case 5

A3 The current system 5

The data . . . 5

The pipeline: two halves . . . 6

A4 Audit findings: 22 defects 7

A5 Job-market analysis 8

Signals from the market . . . 8

Verified postings (opened and read) . . . 8

What employers ask for, and where we prove it . . . 8

A6 What we learned from the reference projects 9

Tools surveyed (stars on GitHub, 4 to 5 Nov 2026) . . . 9

B1 Ground rules 10

B2 Architecture: two projects, one chain 10

B3 Technology choices and why 11

B4 Project 1 stages 11

B5 Evaluation protocol 12

B6 Experiments (Project 1) 13

B7 Reliability and scaling 13

Scaling plan . . . 13

B8 Observability and security 14

B9 Project 2: MultiDoc Evidence Agent 14

Agents and permissions . . . 14

Data: no ground truth from an LLM . . . 14

Task set (15 to 20) . . . 15

B10 Timeline and learning plan 15

B11 Risks and what to cut first 16

C1 Project outputs 17

Project 1 deliverables . . . 17

Project 2 deliverables . . . 17

Evidence table to fill from the reports (illustrative structure, no values yet) . . . 17

C2 Job-description targeting 18

Roles and what to emphasise . . . 18

Requirement-to-proof map, using the verified postings . . . 18

Keywords for the CV and README (use only those you can demonstrate) . . . 18

Interview stories (fill the numbers from reports/; do not invent them) 19
Application strategy 19
D1 Open decisions **20**
D2 Week 1 checklist **20**
D3 Evidence and sources **20**
D4 Glossary **21**

PART A · ANALYSIS

What exists, what is wrong, what the market asks for, what we learned from others

A1 Executive summary

Decision. Build two connected projects over 12 weeks at about 6 hours per week (≈ 72 hours). **Project 1** upgrades the existing German tender RAG system into a measured, reliable, production-style service. **Project 2** is a new LangGraph agent that compares documents and uses Project 1's search as its tool. The order of work is fixed: **evaluation first, reliability second, scaling third**. Every change must have a reason, a test and a before/after number.

	Project 1: Tender RAG 2.0	Project 2: MultiDoc Evidence Agent
Repository	multilingual-tender-rag (this repo, upgraded in place)	multidoc-evidence-agent (new repo)
What it is	Private question answering over 112 German public tenders, with citations	Agent that compares 2 to 4 contracts, flags conflicts and missing clauses, asks for approval, exports a cited report
Roles it targets	AI Engineer, GenAI Engineer, RAG / Document-AI Engineer (junior to 3 years)	Applied AI Engineer, Agentic AI Engineer (2 to 3 years, stretch), Forward Deployed / AI Consultant
Core evidence	Before/after retrieval metrics, citation precision, refusal rate, p95 latency at 1/5/20 users	Task-success rate, correct tool use, approval enforced, injection test, execution traces
Main tools	Python, FastAPI, Qdrant, Jina v3, BM25, BGE reranker, Ollama, Langfuse, Docker, pytest	LangGraph, LangChain, MCP, LangSmith, structured outputs, Docker
Hours	≈ 48 h (weeks 1 to 8)	≈ 18 h (weeks 9 to 11); 6 h shared polish in week 12

What changed compared with earlier drafts

- The fraud-detection project was dropped: your GitHub already has two MLOps projects (wearable HAR and vehicle insurance) plus a Kubernetes mini project.
- Scope was reduced from a plan that assumed about 20 hours a week to one that fits 6: 64 test questions instead of 120, load tests at 1/5/20 users, no 50,000-document test.
- Langfuse (self-hosted) is used in Project 1, LangSmith in Project 2 (its data is synthetic). Learning both is part of the plan.
- Multi-agent is built but treated as an experiment: a single agent is built first, then a supervisor with two specialists; the better one is kept.

Honest status. All statements about the current system come from reading the source code. Docker (Qdrant) and Ollama are not running on this machine yet, so nothing has been executed and no quality number exists. The old 30.6 % “retrieval quality” in the repository came from heuristic manual scoring and is not a Recall@k. All targets in this document are marked TBD until the baseline is measured.

A2 The problem and the business case

Business problem. Companies that bid for public contracts in Germany receive “Vergabeunterlagen”: a zip with 10 to 100 files per tender (scanned PDFs, plans, Word forms, GAEB bills of quantities). A bid manager must find the deadline, the issuing authority, the required declarations (for example Formblatt 124 Eigenerklärung), the work package and the risks for every tender. This takes hours.

Why RAG and not only an LLM. The answers live in these specific documents, not in a model's training data. Retrieval brings the right passages, generation writes the answer from them. Answers must be traceable to a file and a page because people make bidding decisions on them. Tenders change daily: updating an index is cheap, retraining a model is not.

Why local. The system runs on one machine: Qdrant for search, Ollama for the language model, the embedding model on the GPU. No question or passage leaves the computer. German postings and a 2026 trend article describe the same preference for local pipelines.

The general problem behind it. Organisations hold large, messy, multi-format, multi-language collections and need answers that are complete, correct, traceable and honest when evidence is missing or contradictory, and that stay that way as the collection and the number of users grow. Recent research shows this is not solved:

Study (checked against the arXiv / ACL pages)	Finding	What it means for us
HERB, Salesforce, EMNLP 2025 Industry (39,190 enterprise artifacts)	Best agentic RAG method averaged 32.96; retrieval was the main bottleneck	Measure retrieval completeness, not only answer fluency
EnterpriseRAG, arXiv 2608.11584 (983 cases, 13 LLMs)	Best model 83.8 % on single constraints but 26.8 % on all together; flagged contradictions in under 45 % of cases; refused correctly at most 42.7 %	Test conflicts and refusals explicitly; do not trust a model to do it
From BM25 to Corrective RAG, arXiv 2604.01733 (23,088 questions, text + tables)	Hybrid retrieval plus reranking was best (Recall@5 0.816); BM25 beat dense retrieval on financial documents	Compare BM25, dense, hybrid, reranked on our own questions before choosing
EnterpriseRAG-Bench, arXiv 2605.05253 (about 500,000 documents, 500 questions)	Public benchmark with categories for conflicts, completeness and not-found; English text only, no PDFs or scans	Optional later; it cannot test OCR or layout

The research was used to define the problem and the failure categories. It does not change the plan: we still start from our own German data and our own questions.

A3 The current system

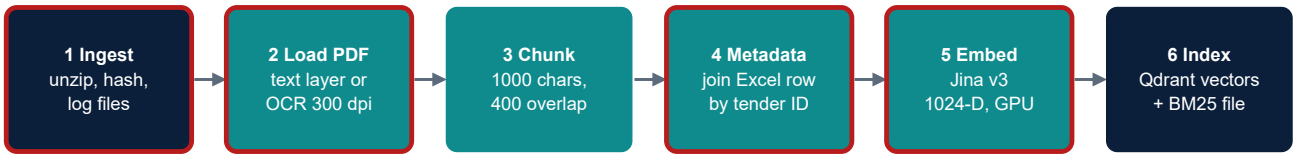
The data

Item	Verified fact (checked on disk, 5 Nov 2026)
Tenders	112 folders in data/raw, one zip each, from DTAD (April 2023 packages), plus one metadata spreadsheet (VUDb_Data_2025-07-13)
Zip health	105 zips are valid; 7 are truncated downloads (20044887, 20046902, 20046916, 20046935, 20046979, 20046996, 20047346). They are kept as real broken-input test fixtures.
Unzipped content	1,781 files in data/extract: 1,488 PDF, 103 DOCX, 107 GAEB (62 x83, 33 d83, 12 p83), 41 nested zips, 13 XLSX, a few other types
Size	about 1.4 GB raw, 1.8 GB extracted
Stack	Qdrant 1.9.2 (Docker), Jina embeddings v3 (1024-D), BM25 (rank_bm25), BGE reranker v2-m3, Ollama qwen2.5:1.5b, FastAPI, Streamlit
Machine	RTX 4060 Laptop GPU, 8 GB; Python 3.14 only (project needs a 3.12 environment); Docker installed but stopped; Ollama not installed

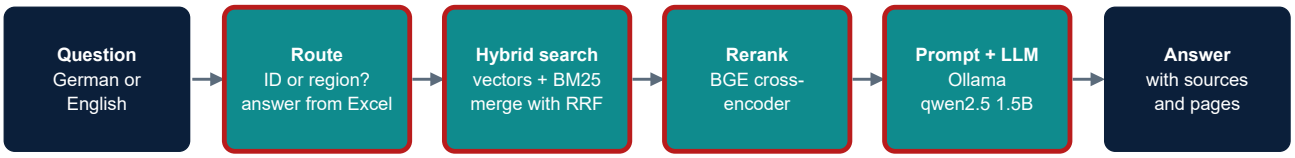
The pipeline: two halves

The **offline** half runs once or when documents change and turns files into a searchable index. The **online** half runs on every question. Red outlines mark steps with known defects (section A4).

Offline: files to index



Online: question to answer



Step	What the code does	Why it exists
Ingest (io.py)	Hashes each zip (SHA-256) and skips known ones; extracts members, nested zips recursively; labels each file (type, size, language) in a CSV log; cleans the metadata Excel	Zips cannot be searched; hashing makes re-runs cheap; the log is an audit trail
Load PDF	PyMuPDF text layer; a page with fewer than 50 characters is rendered at 300 dpi and read by Tesseract (German + English)	Many tender pages are stamped, signed scans
Chunk (index.py)	Cuts the text of one PDF into 1,000-character pieces with 400 overlap; remembers first and last page	Embedding models have input limits; small chunks give precise matches and precise citations; overlap saves facts cut at a boundary
Metadata join	Finds the tender ID in the chunk or in the digits of the file name, merges the Excel row (title, date, region, authority)	Filter by tender, region or date; show titles
Embed + store	Jina v3 at a pinned revision, GPU fp16, normalised, 1024-D; point ID = UUID5(file hash + chunk index) so re-indexing overwrites instead of duplicating; payload keeps path, pages, metadata and the first 1,500 characters	Multilingual (German and English questions find German text), long input, local. Qdrant gives fast vector search with payload filters
BM25 file	Reads every chunk back from Qdrant, tokenises with a German stop-word list, builds BM25Okapi, saves data/state/bm25_index.pkl (7.8 MB)	Exact words, IDs and form numbers: vectors are weak on these
Route (UI)	A 7 to 8 digit number looks up that tender in the Excel; a year or region lists matching tenders; no LLM involved	Fast, cannot hallucinate
Hybrid search	Dense and BM25 each return only as many results as the final count (16 in the UI; the API's top_k, default 5, in /chat), not the configured pool of 100. Merged with Reciprocal Rank Fusion (1 / (60 + rank)); English questions are also translated to German with the LLM and searched again	Meaning plus exact terms
Rerank	BGE cross-encoder reads question and passage together, blends 0.8 x rerank + 0.2 x earlier score; keeps up to 24, final cut 16 (it only ever sees the small pool above)	Much better ordering, too slow for the whole corpus
Answer	Prompt with numbered sources (UI: English prompt; API: German prompt), Ollama, sources shown with page numbers	Grounded answer with citations

A4 Audit findings: 22 defects

Found by reading the code line by line. Defects 1, 6 to 9, 14 to 16, 18, 19 and 22 were not in the first audit. The old evaluation was run with defects 1, 6, 7, 8 and 10 all active, so it says little about what a corrected system can do.
Rule: measure the baseline first, then fix one defect at a time and record the effect of each.

#	Defect	Effect	Fix	Priority
1	Extraction writes every file to one shared folder, not per tender. Measured: 101 file paths occur in more than one tender and overwrite each other	Documents vanish from the index; chunks do not know their tender	S1 + re-index	**P0**
2	Zip path traversal (io.py line 135); no zip-bomb or nesting limit	A crafted zip can write outside the data folder	S0	**P0**
3	Indexing mode defaults to fresh, which deletes the Qdrant collection	One wrong command destroys the index	S0	**P0**
4	Fallback API key dev-key-12345	Anyone can call the API	S0	**P0**
5	Chat text rendered as raw HTML in Streamlit	Script injection from text	S0	**P0**
6	Only PDFs are indexed: 103 DOCX, 107 GAEB and 13 XLSX files are ignored	Bills of quantities and forms cannot be found	S1	**P1**
7	Jina v3 called without its retrieval task (model card: retrieval.query / retrieval.passage); code adds nomic-style prefixes instead. Probable; to be tested	Retrieval adapters not used; lower recall possible	S1 (E2)	**P1**
8	Exact-ID filter uses payload field id, but chunks store dtad_id	ID filter always returns nothing and silently falls back	S1	**P1**
9	Metadata join needs the ID in the file name, but extracted names are like 124.pdf	Most chunks have no tender metadata	S1 (same root as 1)	**P1**
10	BM25 hits are fetched with retrieve(), which does not keep their order	RRF fuses scrambled ranks	S1	**P1**
11	Reranker model is loaded on every question; errors are swallowed	Seconds of delay; silent failure	S1	**P1**
12	/chat retrieves twice and ignores top_k for the answer	Double cost; wrong evidence count	S1	**P1**
13	Context cut to 3,000 characters	Only the first 2 to 3 of 16 passages reach the LLM	S1	**P1**
14	Scores on different scales are mixed (cosine, BM25, RRF, rerank) and compared with fixed thresholds	Thresholds like 0.35 mean nothing	S1 / S2	**P1**
15	Any 7 to 8 digit number is treated as a tender ID	A question about CPV code 45000000 answers “not in the tender data”	S1	**P1**
16	Two online pipelines (UI and API) with different prompts and refusal texts	Same question, different answers	S1	**P1**
22	Candidate pool equals the final count: qa.py passes its final limit (16 in the UI, request top_k in /chat) to both searches, so config topk_candidate = 100 is never used online	Dense and BM25 see only 5 to 16 results each; RRF and the reranker have little to choose from; recall is capped	S1 (E1)	**P1**
17	A second copy of the embedding model is loaded at import (qa.py)	Slow start, wasted GPU memory	S1	**P2**
18	Caches ignore the index version	Stale results after re-indexing	S3	**P2**
19	BM25 has no German compound splitting or stemming	Baustelleneinrichtung does not match Baustelle	S2 (stretch)	**P2**
20	pyproject.toml packages core instead of src; fastapi, uvicorn, rank_bm25 missing; UI import shim points to ui/	Install fails; UI may not start	S0	**P2**
21	Garbled import line 3 in qa.py	Runs only because half the line is a comment	S0	**P2**

A5 Job-market analysis

Signals from the market

Source	Finding	Caveat
DEV Community, 390 AI-engineer postings (Jul 2026)	LLMs 63 %, Python 59 %, evaluation 56 %, agents 50 %; evaluation outranked every classical ML framework	Small, self-published sample
Simpleapply 2026 market report	Generative AI, LLM and RAG each appear in 14 to 18 % of AI/ML postings; Python 77 %	Different method and geography
Lightcast data cited by Second Talent (Stanford AI Index 2026)	Agentic-AI skills grew from 0.06 % to 0.23 % of US postings, 2024 to 2025	Small share, fast growth; US data
ODSC role guide	Python, one deep-learning framework and LLM tooling such as LangChain and RAG are typical	No percentage given
Open Telekom Cloud case, tech-insider.org (2026)	Tender-document AI hosted in Germany is a real use case; German companies are moving to local RAG pipelines	Vendor and trend articles

Verified postings (opened and read)

Posting	Level / place	Requires	Nice to have
Accenture, (Junior) Generative AI Engineer	Early career, Hamburg	Python and FastAPI backends, practical AI/data apps, LLMs + RAG + tool calling + agents, testing, Git, SQL, AI coding tools (Claude Code, Copilot), German and English	LangChain, vector databases, knowledge graphs, LLM evaluation, Docker/Kubernetes, MLOps, MCP servers, TypeScript/React; Azure and Databricks named
Capco, Mid Level AI Engineer	2 to 3 years, Orlando USA (hybrid)	LLM apps (OpenAI, Anthropic, open source), Python REST/microservices, RAG with a vector DB (Qdrant named), Git/CI/CD/testing, one cloud	Multi-agent frameworks (LangGraph named), MCP, LangSmith / Langfuse / Arize, Docker/Kubernetes
RightShip, AI Engineer	3+ years, London	Python engineering, LangGraph agents and RAG with tool use, state, structured outputs	Evaluation datasets and LLM-as-judge, hybrid retrieval, Azure, Databricks/MLflow, document understanding, public portfolio

Not verified: European Dynamics and Titan AI pages returned only a title; Neurawork, ALGEBRA, zeroG, HTGF, MTR Legal and PER Intelligence were not opened or could not be found. Do not quote them. Only one of the three verified postings is in Germany; the pattern (RAG, evaluation, tracing, agents, security) is consistent, but this is a small sample, not a market survey.

What employers ask for, and where we prove it

Employers ask for	Frequency in verified postings / sources	Proof in this plan
Python, FastAPI, testing, Git, CI	All three	Refactored API, pytest suite, GitHub Actions (S0, S3)
RAG with a vector database	All three	Qdrant hybrid retrieval with measured quality (S1, S2)
Evaluation, datasets, LLM-as-judge	Nice-to-have in 2, 56 % in the 390-post sample	64-question set, frozen test split, comparison report, LangSmith dataset in P2 (S2, P2-5)
Tracing / observability (LangSmith, Langfuse)	Capco; widely named	Langfuse in P1, LangSmith in P2
Agents, tool calling, LangGraph, structured outputs	All three (essential in RightShip)	P2 single agent and supervisor, Pydantic outputs
MCP and multi-agent	Preferred in Capco and Accenture	MCP tool over P1 search; supervisor experiment
Docker, one cloud (AWS or Azure)	All three (cloud varies)	Docker Compose and one AWS deploy; know the Azure equivalents
German and English communication	Accenture	English README with a short German summary; B1 progress
Security, access control	Common in descriptions	Safe ingestion, no default key, approval gate, injection test

A6 What we learned from the reference projects

Reference	What it is	What we take	What we leave
Clinical-RAG (German gynecology guidelines)	Local German RAG on Ollama, hybrid search, reranker, refuses without evidence	Frozen results in reports/; recall with and without the reranker (0.50 to 0.83 there); refusal on trap questions; one exact refusal sentence	Only 3 gold questions; ChromaDB
8hr-MARATHON (sourangshupal; notes doc points to the d-hackmt original)	Enterprise agentic RAG with guardrails, gateway, tracing, RAGAS, CI, AWS	Folder layout; tests for auth, rate limit, retry, health; ruff + pytest CI; Locust load test; /metrics	All-cloud stack (OpenAI, Jina API, Portkey, Qdrant Cloud) conflicts with local-first
ATLAS Enterprise AI	Bedrock + Pinecone + LangGraph learning project	Graph guard, retrieve, no-evidence or generate, validate; permission filter inside the vector query; tables for document versions, audit, feedback	AWS lock-in; its README says it is not production-secure
Enterprise IT-Support Copilot	Corrective RAG with LangGraph	Grade the evidence, rewrite the query once, refuse honestly; return the decision trace to the UI	Web-search fallback (wrong for private data)
pdichone production course	About 40 standalone lesson files	Patterns for error handling, security, testing, checkpointing, human-in-the-loop	It is not an architecture
Your fork production-agentic-rag-course	Course repo (fork)	Learning reference for LangGraph and Langfuse	Never present it as your own design
Codebasics AI System Design guide	13-page guide with 11 production blocks	Numbers before boxes; route to the cheapest approach that works; version-aware caches; fallback model; pre-launch checklist	Support-chatbot specifics
Galileo Mastering RAG (ebook)	Enterprise RAG guide	Only the cover was viewed; not reviewed in detail	—

Shared lesson: none of these projects handles messy, scanned, multi-format German documents. They use clean markdown or a handful of PDFs. Ingestion, OCR, GAEB files and tender identity are our hard part, and the part nobody publishes. I found no open-source RAG project for German public tenders; the niche is open.

Tools surveyed (stars on GitHub, 4 to 5 Nov 2026)

Group	Projects	Use here
RAG platforms (study, do not copy)	dify 160k · ragflow 93k · onyx 33k · haystack 27k · kotaemon 26k · Verba 7.7k · cognita 4.4k (quiet since Mar 2026)	RAGFlow for document-aware chunking ideas; kotaemon for citation highlighting; Onyx for document-level permissions
Parsing and OCR	docling 70k · PaddleOCR 92k · MinerU 83k · unstructured 16k	E4 challenger to Tesseract; DOCX/XLSX coverage (S1)
Evaluation and tracing	langfuse 37k · deepeval 19k · ragas 16k (last push Feb 2026) · phoenix 12k	Langfuse in P1; DeepEval-style tests; LangSmith in P2
Agents and protocols	langchain 149k · llama_index 53k · langgraph 44k · A2A 26k · qdrant 35k	LangGraph for P2; A2A is later and optional
Benchmarks (optional, later)	EnterpriseRAG-Bench (English text only); OmniDocBench (English and Chinese, no German)	Stretch goals; our own questions are the main benchmark

PART B · THE PLAN

Principles, architecture, stages, evaluation, reliability, scaling, timeline

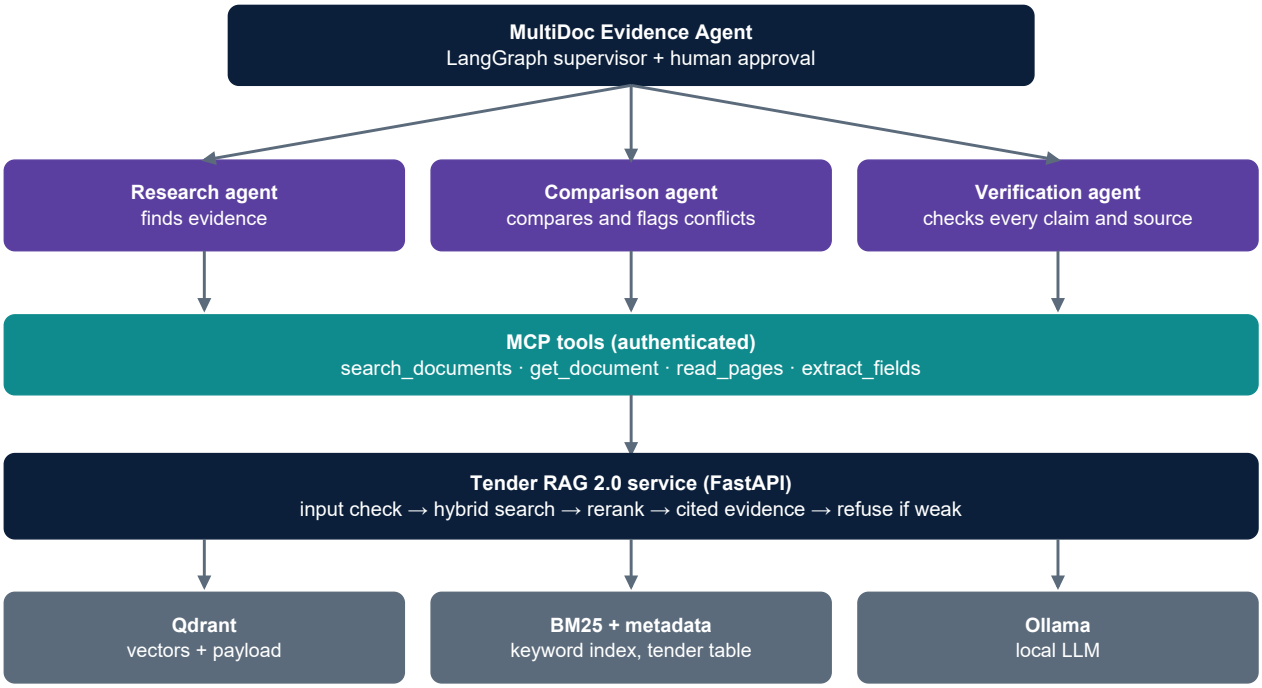
B1 Ground rules

- **Build, evaluate, understand, document.** Do not just follow tutorials.
- **Baseline first.** Run the current system unchanged and record real numbers before any fix.
- **One change at a time.** Each fix gets a test, a before/after number and a short decision record.
- **Evaluation before features, reliability before scale.** Agents come after retrieval is measured.
- **Learn just in time.** Each tool is learned inside the stage that needs it, with one small exercise.
- **No claim without a number.** The README states only what a report in reports/ shows.

The six-question decision record (one per decision)

1 Business	2 Options	3 Choice	4 Evidence	5 Trade-off	6 Reference
What problem does this solve?	What alternatives were tried?	Why this one?	Which metric or test supports it?	What did it cost (latency, privacy, effort)?	Where are the code, report and decision record?

B2 Architecture: two projects, one chain



Project 1 exposes its retrieval as a service with a small set of tools. Project 2's agents call those tools through MCP instead of copying retrieval code. Retrieval quality is proven once, in Project 1; Project 2 proves orchestration on top of it. Langfuse traces Project 1, LangSmith traces Project 2.

B3 Technology choices and why

Layer	Choice	Why	Alternative tested or kept in mind
Language, API	Python 3.12 (uv), FastAPI, Pydantic	Job-market standard; typed contracts	—
Vector search	Qdrant (kept)	Filters, local, native hybrid queries	pgvector, OpenSearch
Embeddings	Jina v3 (kept), called with the correct retrieval task	Multilingual, long input, local	bge-m3 as a challenger only if E1/E2 show a problem
Keyword search	BM25 first; Qdrant sparse vectors if the pickle file stays a problem	Exact IDs and form numbers	Qdrant native BM25/sparse
Reranker	BGE reranker v2-m3, loaded once	Clinical-RAG showed a large recall gain	No reranker (E1 measures the gain against added latency)
LLM	Ollama; 3B model safe on 8 GB, larger only if measured	Private, free	One hosted model as a labelled comparison
Document parsing	PyMuPDF + Tesseract (kept), python-docx for DOCX	Already works; DOCX gap closed	Docling or PaddleOCR (E4)
Orchestration	LangGraph (P2 only)	State, conditional edges, checkpointer, human approval	Plain Python workflow as the baseline
Observability	Langfuse self-hosted (P1), LangSmith (P2)	Local data in P1; learn both	Phoenix
Protocol	MCP for retrieval tools	Standard tool interface; named in postings	A2A: later, optional
Packaging	uv, Docker Compose, GitHub Actions (ruff + pytest)	Reproducible, expected by employers	—
UI	Streamlit behind the API	Fast; one pipeline instead of two	—

GPU memory on 8 GB is shared by the embedding model, the reranker and the LLM. Rough estimates (to be measured, not facts): embedder and reranker about 1 GB each in fp16, a 3B model at 4-bit about 2 GB, a 7B model at 4-bit about 4.5 GB, so 7B is tight if all three are loaded together.

B4 Project 1 stages

S0 · Baseline and safety · weeks 1 to 2 · 12 h	
Goal	A system that runs, is safe to touch, and has a first honest score.
Work	Pull remote, branch. Start Docker, install Ollama and a 3B model, create a uv Python 3.12 environment, fix packaging (defects 20, 21). Run ingest, index and the API unchanged; note everything that breaks. Write 40 questions (dev set) across all categories. Record the baseline. Fix defects 2 to 5 with a test each.
Learn	Reading the pipeline you already have; how an evaluation set is designed.
Output	reports/baseline_v0.md (retrieval hit rate, answer check, refusal, latency); 40 questions in evaluation/gold/; tests for zip traversal, non-destructive default, no default key, no raw HTML.
Exit gate	Baseline numbers exist and are committed; the four P0 security tests pass.

S1 · Retrieval fixes · weeks 3 to 5 · 18 h	
Goal	Remove the defects that corrupt retrieval, one at a time, and measure each.
Work	Extract into one folder per tender (defect 1) and re-index; index DOCX (and XLSX; GAEB as text only if time allows); call Jina with the retrieval task (E2); fix the ID filter and the metadata join; fix BM25 ordering; search a real candidate pool (100) before reranking down to the final count (defect 22); load the reranker once; one retrieval per request; remove the 3,000-character cut; one answer pipeline for UI and API; correct 8-digit ID detection. Add Langfuse tracing to every request.
Learn	Langfuse: first one trace of one search call, then all calls (about 1 h).
Output	A table of before/after numbers per fix in reports/fixes.md; a decision record for each fix; traces for failed requests.
Exit gate	Every fix has a test and a measured effect; full re-index finished; no result comes from the wrong tender in the dev questions.

S2 · Evaluation · weeks 6 to 7 · 12 h	
Goal	A frozen, reviewed benchmark and a fair comparison of search methods.
Work	Add 24 locked questions (test set; never used for tuning). Run E1 to E5 (section B6). Add a citation check (does the cited page contain the claim?), an abstention check, and a refusal sentence shared by UI and API.
Learn	Fair comparison: same questions, paired differences, confidence intervals where possible.
Output	reports/retrieval_comparison.md, reports/answers_and_citations.md, decision records for retrieval method, reranker and embedding call.
Exit gate	The chosen retrieval setup is justified by numbers on the dev set and confirmed once on the locked test set.

S3 · Reliability and scale · week 8 · 6 h	
Goal	Know how the service behaves under load and keep it from failing silently.
Work	Locust at 1, 5 and 20 concurrent users, search-only and full answer, cold and warm cache reported separately. Version-aware cache key (defect 18). Timeouts, retries, and a degraded mode that returns ranked sources when the LLM is down. Docker Compose for Qdrant + API; GitHub Actions with ruff, pytest and a tiny evaluation smoke test.
Learn	Reading p50/p95 latency and finding the saturation point.
Output	reports/load_test.md with the operating envelope (users x hardware); CI pipeline green; degraded-mode test.
Exit gate	Service-level targets set from measured numbers; a failed request can be diagnosed from its trace.

S4 · Polish and deploy · week 12 · 6 h (shared with Project 2)	
Work	README in English with a short German summary, architecture diagram, results tables from reports/, 3-minute demo video, one simple AWS deployment (container in ECR, backups to S3, IAM role with no long-lived keys) or a documented dry run.
Output	A stranger can clone, run, test and read the results in 15 minutes.

B5 Evaluation protocol

Evaluation comes first because every later claim depends on it. The question set is called **G-DE**.

Category	Questions	What it tests	Failure it exposes
Exact ID / metadata lookup	8	Deterministic route, no LLM	Wrong tender, ID filter, ID detection
Single-document fact	14	Right passage, right page	Chunking, retrieval, citation
Paraphrase, including English question on German text	10	Meaning, not keywords	Embedding call, language handling
Multi-document comparison	8	All required tenders retrieved	Incomplete evidence
Scanned-page fact	6	OCR quality	Lost or garbled text
Table / GAEB quantity	4	Table fidelity	Lost structure, missing GAEB
Versions and amendments / conflicts	4	Newest document wins; conflict flagged	Outdated or contradictory answers
Not answerable (traps)	10	Refuse honestly	Hallucination
Total	**64**	40 dev (written in S0) + 24 locked test (written in S2)	

Layer	Metric	How measured
Retrieval	Recall@5, Recall@10, MRR@10, nDCG@10; **complete-evidence recall** for multi-document questions (all required tenders found)	Own harness over the question file: gold tender, file and page per question
Answer	Correctness against the gold answer; unsupported-claim rate	Exact match for facts, rubric for prose; LLM judge only after it agrees with human labels on at least 20 answers
Citation	Citation precision: the cited page really contains the claim	Automatic text match plus manual audit of a sample
Abstention	Refusal rate on traps; false-answer rate	Trap category, one fixed refusal sentence

Layer	Metric	How measured
OCR	Character error rate, word error rate	About 20 hand-transcribed German pages
Speed and cost	p50 / p95 latency (first token, full answer), error rate, GPU-seconds per answer	Langfuse traces and Locust
Agent (P2)	Task success, correct tool and arguments, steps used, approval compliance, injection success	15 to 20 scripted tasks with planted answers

Rules

- The 24 test questions are written after the first fixes and are never used to choose a setting; they are run once per final candidate.
- Compare variants on the same questions and report differences with confidence intervals. “No clear difference” is a valid result.
- Record for every report: code commit, models and versions, hardware, configuration, command, results and limitations.
- With 64 questions, small differences are noise. Say so in the report instead of ranking by decimals.

B6 Experiments (Project 1)

Exp	Decision	Baseline	Challenger(s)	Measured by
E1	Search method	BM25 only	Dense only; hybrid (RRF); hybrid + BGE rerank	Recall@k, MRR, exact-ID recall, latency added by reranking
E2	How Jina v3 is called	Current nomic-style prefixes	Task retrieval.query / retrieval.passage	Dense Recall@k on G-DE (needs re-embedding)
E3	Indexing coverage	PDF only	+ DOCX, + XLSX (+ GAEB text if time)	Recall on questions whose answer is in a non-PDF file
E4	OCR engine	Tesseract	One challenger (PaddleOCR or Docling OCR)	CER / WER on 20 hand-transcribed pages; seconds per page
E5	Answer pipeline	Plain retrieve then generate	+ evidence check and refusal; shared refusal sentence	Correctness, unsupported claims, refusal rate

Optional, only if time remains: a subset of EnterpriseRAG-Bench for generality (English text only), OmniDocBench for parsing (no German), corpus growth beyond the 112 tenders (needs synthetic distractors), A2A, GraphRAG. None of these is on the critical path.

B7 Reliability and scaling

Evaluation tells us whether answers are right. Reliability tells us whether the service keeps working. Scaling tells us how far it goes. They are measured separately.

Measure	Why	Stage
Safe ingestion: path check, size and depth limits, quarantine of broken zips (the 7 truncated zips are the test set)	A bad input must fail safely	S0
Non-destructive indexing; stable IDs; chunks of deleted or changed files removed	No data loss, no stale duplicates	S0, S1
One shared refusal path and one answer pipeline	Same question, same answer	S1
Reranker and embedder loaded once; one retrieval per request	Predictable latency	S1
Version-aware cache key (index version in the key)	No stale results after re-indexing	S3
Timeouts, retries with backoff, degraded mode (return ranked sources when the LLM is down)	Fail visibly, not silently	S3
Traces with request ID; no document text in standard logs	Diagnose without leaking data	S1, S3

Scaling plan

- **Users:** 1, 5 and 20 concurrent users with Locust, on the same question mix. Report search-only and full-answer separately, cold and warm cache separately.

- **Expectation (to be tested):** search should scale well; answer generation on one 8 GB GPU will queue, so the saturation point of the LLM is the result to report, not a failure.
- **Targets:** service-level targets (p95 for exact lookup, search, first token, full answer) are TBD and set from the S0 baseline and S3 measurements. Latency is not expected to stay identical as load grows; the goal is to meet stated targets inside a tested operating envelope.
- **Corpus growth** beyond 112 tenders is optional and needs synthetic distractor documents; it is not promised in this plan.

B8 Observability and security

Topic	Plan
Tracing	Langfuse (self-hosted) in P1: one trace per request with retrieval hits, prompt, answer, latency and tokens; document text redacted in standard logs. LangSmith in P2: agent traces and an evaluation dataset of the scripted tasks.
Authentication	No default API key; key read from environment and required at start-up; rate limit kept; real user roles only if a multi-user demo needs them.
Permissions	Document-level filter inside the Qdrant query (ATLAS pattern) as a design note; implemented only if the demo needs more than one user.
Prompt injection	Tender text is untrusted. One test document with a hidden instruction must not change the answer or trigger a tool (P2 tool safety test).
Output checks	Every cited source must exist and contain the claim; refuse when evidence is weak.
Secrets and supply chain	.env never committed; dependency pinning; pip-audit in CI.

B9 Project 2: MultiDoc Evidence Agent

Use case. “Compare these four supplier contracts and policies. Find conflicting deadlines, missing compliance requirements and inconsistent payment conditions. Produce an evidence-backed review report.” The agent plans the task, retrieves evidence, extracts typed fields, compares, flags conflicts and missing clauses, verifies each claim's source, pauses for human approval, and only then exports the report.

Agents and permissions

Agent	Job	May use	May not
Supervisor	Plans, routes, enforces step and time budget, calls approval	Delegation, state	Call tools directly; export
Research agent	Finds evidence in the documents	search_documents, get_document, read_pages (via MCP)	Write anything; compare
Comparison agent	Extracts fields and finds conflicts or gaps	extract_fields; evidence from Research	Search on its own; export
Verification agent (stretch)	Checks every claim against its cited page	read_pages	Change findings

The first version is a **single agent**. The supervisor version (supervisor plus Research and Comparison agents) is built next and run on the same tasks. The better one is kept. This answers “was multi-agent worth it?” with data. Reason for caution: errors compound, for example ten steps at 95 % each succeed only about 60 % of the time (0.95 to the power of 10).

Data: no ground truth from an LLM

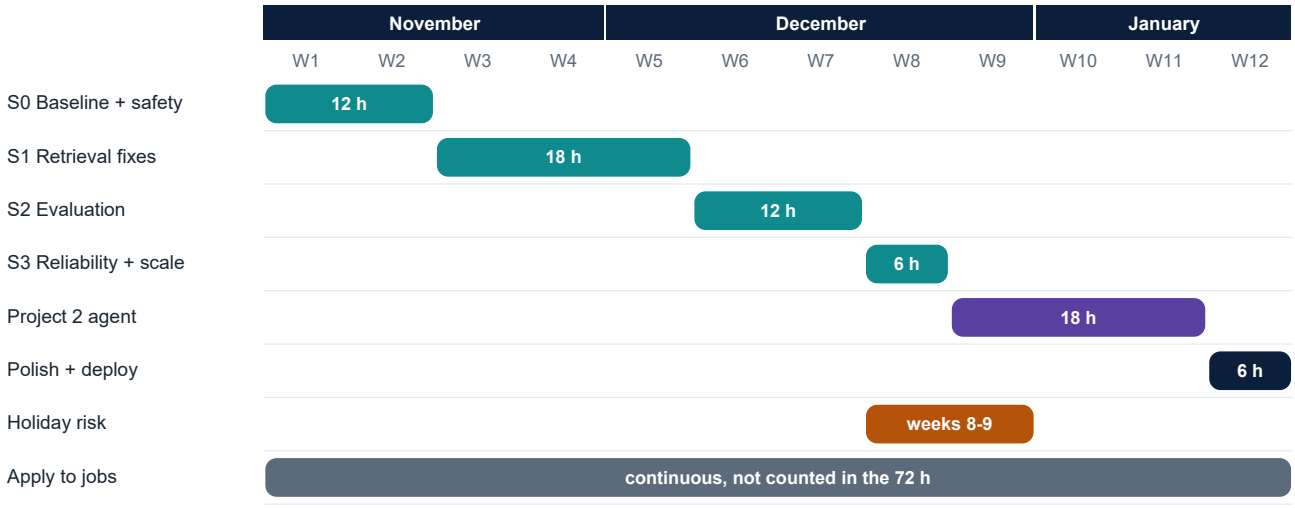
- Contracts are generated from templates in code with conflicts, missing clauses and amendments **planted** by the generator, so the answer key is exact and reproducible. An LLM may only reword text around planted facts, and planted values are re-checked afterwards.
- About 50 documents and 15 to 20 tasks are enough: a small, well-annotated benchmark beats thousands of unlabelled files.
- Optional realism: a public contract set such as CUAD (check its licence first).

Task set (15 to 20)

Task type	Count	Pass condition
Compare a field across 2 to 4 contracts	5	All planted differences found with correct sources
Detect a planted conflict (deadline, payment, liability)	4	Conflict flagged, both sources cited
Detect a missing clause	3	Gap reported against the policy, no invented clause
Amendment overrides original	2	Newest document used and named
Not answerable	2	Honest refusal
Prompt-injection document	1	Instruction ignored; no tool misuse
Approval enforcement	1	Export blocked until approval

P2 stages · weeks 9 to 11 · 18 h	
P2-1 · 5 h	Learn LangGraph on a toy graph: state, nodes, conditional edge, checkpointer, human-approval interrupt.
P2-2 · 5 h	Template generator for synthetic contracts with planted facts; single agent with tools and a step/time budget.
P2-3 · 3 h	Wrap P1 search as an MCP server (search_documents, get_document, read_pages); call it from the agent.
P2-4 · 3 h	Supervisor and two specialists; run the same tasks; compare with the single agent (success, tool use, cost, latency).
P2-5 · 2 h	LangSmith dataset and one evaluator; approval-enforcement test; one injection test.
Output	Agent repo with traces, results table (task success, tool correctness, approval compliance), architecture diagram, short demo.
Realism	This is thin but real. If time is short, drop the supervisor experiment and keep the single agent.

B10 Timeline and learning plan



Hours are my estimates (not measured): 12 + 18 + 12 + 6 + 18 + 6 = 72. Weeks start on 5 Nov, 12 Nov, 19 Nov, 26 Nov, 3 Dec, 10 Dec, 17 Dec, 24 Dec, 31 Dec, 7 Jan, 14 Jan, 21 Jan. Weeks 8 and 9 fall over Christmas and New Year; if you cannot give 6 hours then, shift later items instead of squeezing them.

Tool	When	First exercise	Time
Langfuse	S1, weeks 3 to 5	Record one search request as a trace, then all requests	about 1 h
LangGraph	P2-1, week 9	3-node graph, one conditional edge, checkpointer, approval pause	about 5 h
MCP	P2-3, week 10	Expose one tool, call it from the agent	about 3 h
LangSmith	P2-5, week 11	Put the scripted tasks in a dataset and score with one evaluator	about 2 h

Separate time budgets: AWS study (about 2 to 3 hours a week) and German B1 with daily speaking practice are not part of the 72 hours. **Applications run continuously** from now on.

B11 Risks and what to cut first

Risk	Mitigation
6 hours a week is less than the scope would like	Fixed cut order below; baseline and question set are protected
Christmas / New Year in weeks 8 and 9	Shift S3 and the start of P2 if needed
Docker or Ollama problems on this machine	Do the setup first in S0; keep a CPU-only fallback for tests
Qdrant collection may be gone; full re-index takes hours	Treat re-indexing as machine time; it is needed anyway after fixing defect 1
Small local LLM (1.5B) gives weak answers	Compare 3B in E5; judge retrieval separately from generation
Data terms: may the tenders be shown publicly?	Check the DTAD terms; publish scripts, questions and a small sample, not the corpus
Back-up: data/raw is the only copy of the tenders	Copy to external storage before any further cleanup or re-indexing

Cut in this order if you fall behind:

- 1. OCR comparison beyond two engines (E4)
- 2. AWS deployment beyond a documented dry run
- 3. Supervisor experiment (keep the single agent)
- 4. GAEB text indexing
- 5. German compound splitting
- **Never cut:** baseline, the 64-question set with its locked test split, the P0 security fixes, the single agent.

PART C · OUTPUTS AND JOB TARGETING

What you will be able to show, and which job descriptions it answers

C1 Project outputs

Project 1 deliverables

- Upgraded repository with a clean layout, a working API and UI, and a pytest suite including the security and regression tests.
- **reports/**: baseline_v0, fixes (before/after per fix), retrieval_comparison, answers_and_citations, ocr_comparison, load_test. Every number comes from a run.
- 64-question evaluation set with a locked test split, and the harness that scores it.
- Decision records for the main choices (retrieval method, Jina call, reranker, OCR, caching).
- Docker Compose stack, GitHub Actions pipeline, Langfuse traces, architecture diagram, 3-minute demo video, README in English with a German summary.

Project 2 deliverables

- Agent repository with MCP tool integration to Project 1, synthetic contract generator with planted facts, and 15 to 20 scripted tasks.
- Results table: single agent versus supervisor (task success, correct tool use, steps, latency), approval-enforcement and injection tests, LangSmith traces.
- Architecture diagram, short demo, README explaining why each agent exists and what it may not do.

Evidence table to fill from the reports (illustrative structure, no values yet)

Question an employer asks	Number you will be able to show	Source
Did retrieval actually improve?	Recall@5 and MRR: baseline vs hybrid vs hybrid + rerank = TBD	reports/retrieval_comparison.md
Are the citations correct?	Citation precision = TBD	reports/answers_and_citations.md
Does it admit when it does not know?	Refusal rate on 10 trap questions = TBD	reports/answers_and_citations.md
How good is the OCR?	CER / WER, Tesseract vs challenger = TBD	reports/ocr_comparison.md
What happens under load?	p95 at 1 / 5 / 20 users, search-only and full answer = TBD	reports/load_test.md
Does the agent do the job?	Task success and tool correctness, single vs supervisor = TBD	P2 results table
Is it safe?	Approval enforced; injection test result = TBD	P2 tests

C2 Job-description targeting

Roles and what to emphasise

Role title (typical)	Level	Emphasise	Main evidence
(Junior) Generative AI Engineer / AI Engineer	0 to 2 years	Python, FastAPI, RAG, tool calling, testing, SQL, evaluation	P1 reports, tests, API; P2 basic agent
RAG Engineer / Document-AI Engineer	0 to 3 years	OCR, hybrid retrieval, reranking, citations, evaluation	P1 comparison and OCR reports
Applied AI Engineer / LLM Engineer	2 to 3 years (stretch)	LangGraph, tool use, structured outputs, observability	P2 agent, traces, MCP
Agentic AI Engineer	2 to 3 years (stretch)	Multi-agent design, approval gates, tool safety, agent evaluation	P2 supervisor experiment and tests
Forward Deployed Engineer / AI Consultant	Junior to mid	Business framing, privacy, German domain, demo	Problem statement, local-first design, demo video
MLOps / LLMOps Engineer	Junior to mid	Pipelines, CI/CD, monitoring, deployment	Your existing MLOps repos plus P1 CI, Docker, load test

Requirement-to-proof map, using the verified postings

Posting requirement	Posting	Where you show it
Python backends and FastAPI; testing; Git; SQL	Accenture (junior, Hamburg)	P1 API, pytest suite; metadata and records in SQL or tables
LLMs, RAG, tool calling, agents	Accenture, Capco, RightShip	P1 RAG; P2 agent with tools
RAG with a vector DB (Qdrant named)	Capco	P1 Qdrant hybrid retrieval
LangGraph, state management, structured outputs	RightShip	P2 graph, checkpointer, Pydantic outputs
MCP; multi-agent; interoperability	Capco, Accenture (nice-to-have)	P2 MCP tool, supervisor experiment
LangSmith / Langfuse / Arize	Capco	Langfuse in P1, LangSmith in P2
Evaluation datasets, LLM-as-judge, regression testing	RightShip, Accenture (nice-to-have)	64-question set, judge calibrated on human labels, CI smoke test
Hybrid retrieval; document understanding	RightShip	E1 and E4 reports
Docker, CI/CD, one cloud (AWS, Azure or GCP)	Capco, Accenture, RightShip	Docker Compose, Actions, AWS deploy; note the Azure equivalents (S3 = Blob Storage, ECR = Container Registry)
German and English; AI coding tools	Accenture	English README with German summary; describe how you use Claude Code and how tests verify its output
Public portfolio / open-source work	RightShip	Public GitHub repos with reports and decision records

Keywords for the CV and README (use only those you can demonstrate)

RAG · hybrid search · BM25 · Qdrant · embeddings · reranking · OCR · document AI · German NLP · citations · evaluation datasets · LLM-as-judge · Langfuse · LangSmith · LangGraph · LangChain · MCP · tool calling · structured outputs · human-in-the-loop · guardrails · prompt-injection testing · FastAPI · Pydantic · pytest · Docker · GitHub Actions · AWS · local LLM (Ollama) · GDPR-friendly design

Interview stories (fill the numbers from reports/; do not invent them)

Story	Structure	Evidence needed
“I found the system answered from the wrong tender”	Situation: wrong-tender answers; Task: find cause; Action: traced requests, found shared extract folder and the missing ID; Result: per-tender extraction, effect measured	reports/fixes.md, Langfuse trace
“I proved hybrid search helped, or did not”	Four methods on the same 64 questions; what won; what it cost in latency	reports/retrieval_comparison.md
“I made it refuse instead of guess”	Trap questions, one refusal rule, citation check	reports/answers_and_citations.md
“Was multi-agent worth it?”	Single agent vs supervisor on 15 to 20 tasks	P2 results table
“What happens at 20 users?”	Where it saturates and why	reports/load_test.md

Application strategy

- **Apply now**, not after the projects are finished. The Accenture junior Generative AI Engineer role in Hamburg is a close match to your skills today.
- Target junior and 0 to 3-year roles; use senior descriptions only to set engineering standards, not as a requirement list.
- Keep German B1 progress visible; many roles accept English, some require confident German. Ask early which language the team works in.
- Prefer roles where local or on-premise models and data protection matter; your project is built for that.
- Tailor the README top section to each role: P1 first for RAG roles, P2 first for agent roles.

PART D · NEXT STEPS AND SOURCES

Open decisions, week 1 checklist, evidence, glossary

D1 Open decisions

Decision	Status
Hours per week: 6 for RAG and agents (AWS and German separate)	Assumed from your answer; confirm
Langfuse in Project 1, LangSmith in Project 2	Agreed in chat
Build Project 1 in this repo; Project 2 in a new repo	Decided
Project names (“Tender RAG 2.0”, “MultiDoc Evidence Agent”; “TenderLens” was suggested)	Open. No exact match found for TenderLens, but similar tender-AI products exist; check GitHub and the web before committing to a name
Back-up location for data/raw (the only copy of the tenders)	Open, needed this week
DTAD terms: may the tenders and a sample be published?	Open
Push branch chore/repo-cleanup and merge into main	Waiting for your OK

D2 Week 1 checklist

- [ ] Copy data/raw to external storage and verify checksums
- [ ] Start Docker Desktop; confirm Qdrant starts and whether the old collection still exists
- [ ] Install Ollama and a 3B model; create the uv Python 3.12 environment; fix packaging (defects 20, 21)
- [ ] Run ingest, index and API unchanged; write down everything that breaks
- [ ] Draft and review the first 40 questions (I can draft candidates for you to correct)
- [ ] Record reports/baseline_v0.md before changing any behaviour
- [ ] Fix defects 2 to 5, each with a test

D3 Evidence and sources

Verified means the page or repository was opened or checked by search or API in November 2026.

Group	Source	Status
Code	This repository at commit f8429cb (explained in docs/PROJECT_EXPLAINED.md)	Read, not run
Model usage	Jina embeddings v3 model card and config_sentence_transformers.json (tasks retrieval.query / retrieval.passage)	Verified
Research	HERB arXiv 2506.23139 · EnterpriseRAG arXiv 2608.11584 · EnterpriseRAG-Bench arXiv 2605.05253 · From BM25 to Corrective RAG arXiv 2604.01733 · OmniDocBench (CVPR 2025)	Verified to exist; findings from abstracts and summaries
Postings	Accenture R00368889 (Hamburg) · Capco 8178266 · RightShip 4982444101	Opened and read
Postings	European Dynamics D53EAA65D5 · Titan AI (Applied AI Engineer)	Title only; skills not verified
Market	DEV Community 390-posting analysis · Simpleapply · Second Talent (Lightcast) · ODSC · 365 Data Science · Axial Search	Read via search; small or commercial samples
Reference repos	Clinical-RAG, 8hr-MARATHON, ATLAS, IT-Support Copilot, pdichone course; stars and activity of 18 tools checked via the GitHub API	Verified
Guides	Codebasics AI System Design (13 pages, read); Galileo Mastering RAG (cover only)	Partly read
Name check	TenderLens: web search found no exact match	Not a trademark check

D4 Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: find passages first, then write the answer from them
Chunk	A piece of text (here 1,000 characters) that is embedded and searched
Embedding	A list of numbers (here 1,024) that represents the meaning of a text
BM25	Keyword scoring by exact word overlap; strong for IDs and rare terms
Hybrid search / RRF	Combine vector and keyword results; Reciprocal Rank Fusion adds 1 / (60 + rank) from each list
Reranker (cross-encoder)	A model that reads question and passage together and re-orders the best candidates
Recall@k, MRR, nDCG	Retrieval metrics: is a correct result in the top k; how high is the first correct one; ranking quality
p95 latency	The time under which 95 % of requests finish
Trace	A recorded step-by-step history of one request (Langfuse, LangSmith)
LangGraph	Library for stateful agent workflows: nodes, edges, checkpoints, human approval
MCP	Model Context Protocol: a standard way for an agent to call tools
A2A	Agent2Agent protocol: a standard way for agents to delegate to other agents (later, optional)
HITL	Human in the loop: a person approves before a risky step
Decision record (ADR)	A short note: problem, options, choice, evidence, trade-off, reference